

ENTREGA FINAL - PARTE 1 y 2

Proyecto de Código

Sistema de Login con Interfaz Grafica

usando Python y Visual Studio Code

CIB-07 - Introduccion a la Ciberseguridad | Modulo 7

Nombre del estudiante:	Cristopher Rivera Ortiz	Fecha de entrega:	13 de agosto del 2026
Docente:	Francisco Lara Acute	Grupo / Seccion:	

Resultado final — así se vera tu programa

Antes de comenzar, aqui puedes ver como lucira el programa cuando lo hayas terminado. Usa estas imagenes como referencia mientras sigues los pasos de la guia.

Pantalla 1 — Ventana de login (estado inicial)

Esta es la primera ventana que aparece al ejecutar el programa. Muestra el titulo, los campos de usuario y contrasena, y el boton de inicio de sesion.

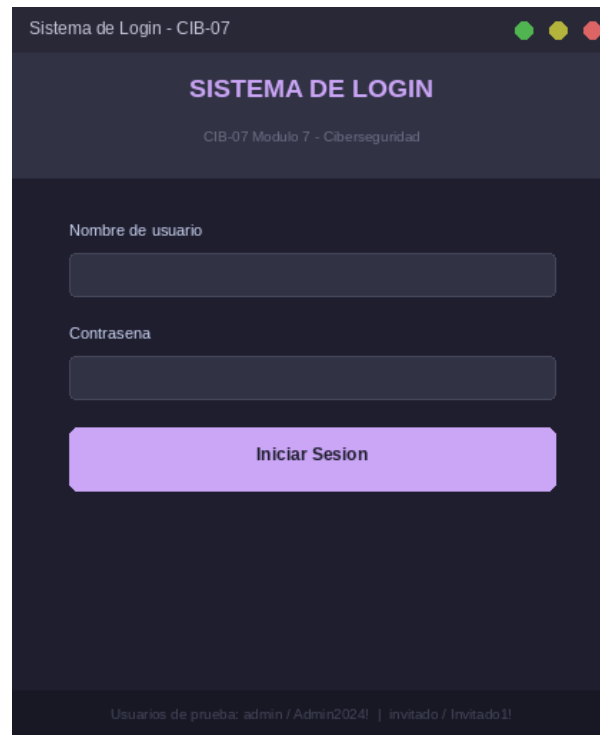


Figura 1. Ventana de login al iniciar el programa.

Pantalla 2 — Ingresando credenciales

Cuando el usuario escribe su nombre y contraseña, el campo activo se resalta con un borde morado. La contraseña aparece como asteriscos para protegerla.

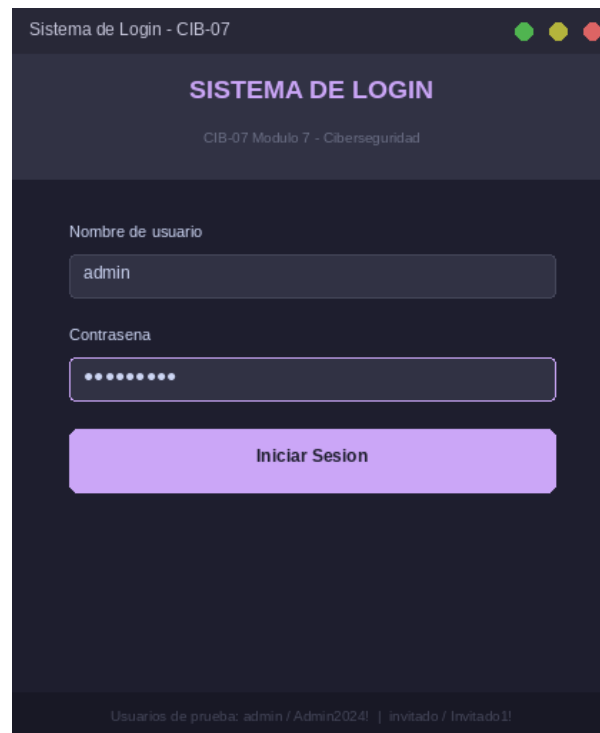


Figura 2. Usuario escribiendo sus credenciales. La contraseña se oculta con asteriscos.

Pantalla 3 — Credenciales incorrectas

Si el usuario ingresa datos incorrectos, aparece un mensaje en rojo indicando cuantos intentos le quedan antes del bloqueo.

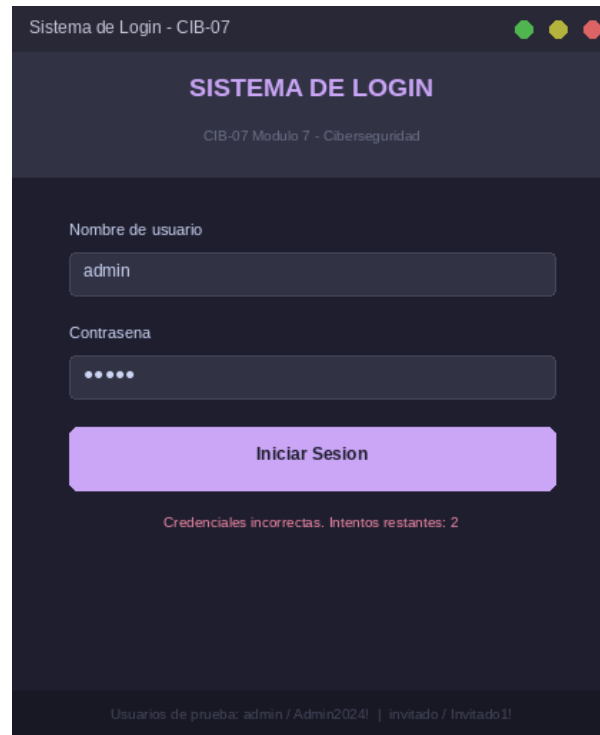


Figura 3. Mensaje de error con contador de intentos restantes.

Pantalla 4 — Acceso bloqueado

Después de 3 intentos fallidos, todos los campos y el botón se desactivan. El sistema muestra un mensaje de bloqueo y ya no permite nuevos intentos.

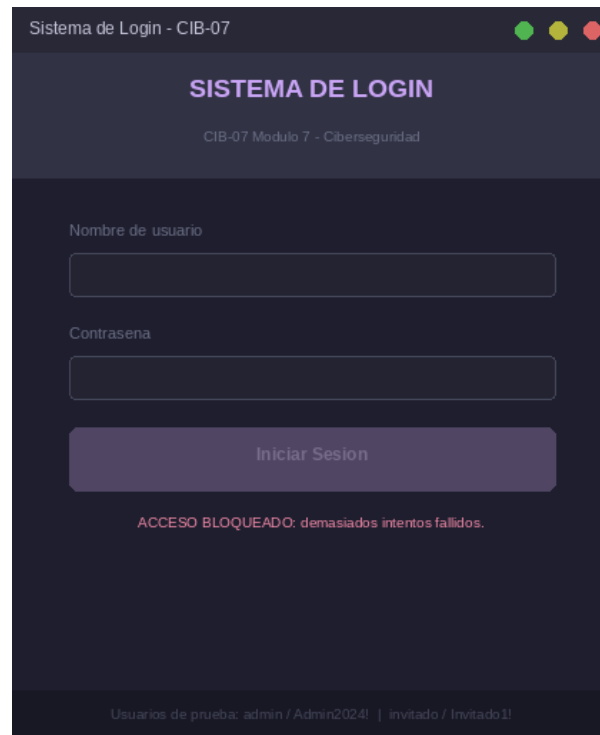


Figura 4. Sistema bloqueado tras agotar los intentos permitidos.

Pantalla 5 — Sesión activa

Cuando el login es exitoso, se abre una nueva ventana con el mensaje de bienvenida, los datos del usuario y el botón para cerrar sesión.

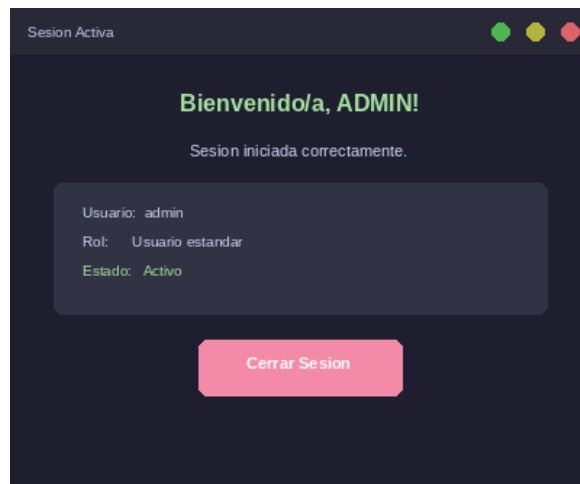


Figura 5. Ventana de sesión activa tras un login exitoso.

Introduccion

En esta entrega vas a construir, paso a paso, un sistema de login con interfaz grafica en Python. El programa mostrara una ventana real con campos de texto, botones y mensajes de error, como los que ves en cualquier aplicacion del día a día.

Este proyecto conecta directamente con los temas del Modulo 7:

- Autenticacion: verificar la identidad de un usuario antes de dar acceso.
- Control de acceso: permitir o denegar el ingreso segun las credenciales ingresadas.
- Principios del SDL: el manejo seguro de contraseñas se decide desde el diseño del programa.
- Ciclo de vida del software: tu programa tiene estructura, logica, manejo de errores y una interfaz.

NOTA: No necesitas experiencia previa programando. Esta guía asume que es tu primera vez y explica cada paso con detalle.

Que vas a construir

Al finalizar esta guía tendrás un programa con ventana grafica que:

- Muestra una ventana de login con campo de usuario y campo de contraseña.
- Oculta la contraseña con asteriscos mientras la escribes.
- Verifica si las credenciales ingresadas son correctas.
- Muestra mensajes de error dentro de la misma ventana si los datos son incorrectos.
- Bloquea el acceso y deshabilita el formulario despues de 3 intentos fallidos.
- Abre una nueva ventana de bienvenida cuando el login es exitoso.
- Permite cerrar sesion y volver a la pantalla de login.

TIP: El programa usa tkinter, una libreria que ya viene incluida con Python. No necesitas instalar nada adicional.

PARTE 1 - Instalacion de herramientas

Antes de escribir codigo necesitas instalar dos programas: Python y Visual Studio Code.

PASO 1 Instalar Python

Descargar Python

1. Abre tu navegador (Chrome, Edge o Firefox).
2. En la barra de direcciones escribe la siguiente direccion y presiona Enter:

```
https://www.python.org/downloads/
```

3. Veras un boton amarillo grande que dice "Download Python 3.x.x". Cualquier version 3.10 o superior funciona.
4. Haz clic en ese boton. Se descargara un archivo .exe a tu carpeta de Descargas.

NOTA: Si tu navegador te pregunta que hacer con el archivo, selecciona Guardar archivo.

Instalar Python

1. Ve a tu carpeta de Descargas y busca el archivo que descargaste. Su nombre empieza con python-3 y termina en .exe.
2. Haz doble clic sobre ese archivo para iniciar la instalacion.
3. Aparecera una ventana de instalacion. **IMPORTANTE:** antes de hacer clic en cualquier boton, busca en la parte inferior una casilla que dice:

```
"Add Python to PATH"    o    "Add python.exe to PATH"
```

4. Marca esa casilla haciendo clic en ella. Debe quedar con una marca de verificacion.
5. Ahora haz clic en el boton "Install Now".
6. Espera a que la instalacion termine. Puede tardar entre 1 y 3 minutos.
7. Cuando aparezca el mensaje "Setup was successful", haz clic en Close.

ATENCION: Si no marcas la casilla "Add Python to PATH" el programa no funcionara correctamente. Si ya instalaste y no la marcaste, desinstala Python desde el Panel de Control y repite el proceso.

Verificar que Python quedo bien instalado

1. Presiona las teclas Windows + R al mismo tiempo. Se abra una ventana pequena que dice Ejecutar.
2. Escribe la palabra cmd y presiona Enter. Se abra una ventana negra llamada "Simbolo del sistema".
3. En esa ventana negra escribe lo siguiente y presiona Enter:

```
python --version
```

4. Deberia aparecer algo como: Python 3.x.x. Si ves eso, Python esta instalado correctamente.

ATENCION: Si aparece un error, intenta escribir python3 --version. Si tampoco funciona, repite la instalacion asegurandote de marcar la casilla PATH.

PASO 2 Instalar Visual Studio Code

Descargar Visual Studio Code

1. En tu navegador ve a:

```
https://code.visualstudio.com/
```

2. Veras un boton azul grande que dice "Download for Windows". Haz clic en el.
3. Se descargara un archivo .exe a tu carpeta de Descargas.

Instalar Visual Studio Code

1. Ve a tu carpeta de Descargas y haz doble clic en el archivo VSCodeSetup que descargaste.
2. Acepta los terminos de la licencia y haz clic en Next.
3. En la pantalla "Select Additional Tasks" marca estas casillas:
 - Add Open with Code action to Windows Explorer file context menu
 - Add Open with Code action to Windows Explorer directory context menu
 - Register Code as an editor for supported file types
 - Add to PATH
4. Haz clic en Next y luego en Install.
5. Espera a que la instalacion termine y haz clic en Finish.

Instalar la extension de Python en VS Code

1. Abre Visual Studio Code desde el icono en tu escritorio o el menu Inicio.
2. En el panel izquierdo haz clic en el icono de cuatro cuadraditos (Extensions) o presiona Ctrl + Shift + X.
3. En el buscador escribe: Python
4. El primer resultado sera "Python" de Microsoft. Haz clic en el boton azul Install.
5. Espera unos segundos. VS Code puede pedirte que recargues la ventana; si lo hace, haz clic en Reload.

PARTE 2 - Tu primer programa en Python

Antes de construir el login completo vamos a asegurarnos de que todo funciona con un programa muy sencillo.

PASO 3 Crear una carpeta para el proyecto

1. Abre el Explorador de archivos de Windows con la tecla Windows + E.
2. Navega a Documentos.
3. Haz clic derecho en un espacio vacío, selecciona Nuevo y luego Carpeta.
4. Nombra la carpeta: login_proyecto

ATENCIÓN: El nombre de la carpeta no debe tener espacios ni caracteres especiales como tildes o la n con tilde.

PASO 4 Abrir la carpeta en VS Code

1. En VS Code haz clic en File (Archivo) y luego en Open Folder (Abrir carpeta).
2. Busca y selecciona la carpeta login_proyecto.
3. Haz clic en Select Folder.
4. Si VS Code pregunta si confías en los autores, haz clic en "Yes, I trust the authors".

PASO 5 Crear y ejecutar tu primer archivo

1. En el panel izquierdo de VS Code haz clic en el icono de nuevo archivo. Nombra el archivo: prueba.py
2. Escribe esto en el editor:

```
print("Hola, mundo!")  
print("Mi primer programa en Python")
```

3. Guarda el archivo con Ctrl + S.
4. Haz clic en el triángulo de "play" arriba a la derecha, o haz clic derecho y selecciona "Run Python File in Terminal".
5. En la parte inferior de VS Code se abrirá la terminal y verás:

```
Hola, mundo!  
Mi primer programa en Python
```

TIP: Si ves ese resultado, todo esta funcionando correctamente. Puedes continuar.

PARTE 3 - Construir el sistema de login

Ahora vamos a construir el programa principal. Lo haremos en fases para que puedas entender cada bloque antes de pasar al siguiente.

PASO 6 Crear el archivo principal

1. En VS Code crea un nuevo archivo dentro de la carpeta login_proyecto. Nombralo: login.py
2. Asegurate de que el archivo este vacio antes de empezar.

PASO 7 Fase 1 - Importar la libreria grafica

Lo primero que debe hacer el programa es cargar tkinter, la libreria que permite crear ventanas y botones en Python.

Escribe esto al inicio del archivo login.py:

```
# =====  
# SISTEMA DE LOGIN CON INTERFAZ GRAFICA  
# CIB-07 Modulo 7 - Ciberseguridad  
# =====  
  
import tkinter as tk  
from tkinter import messagebox
```

Que significa cada linea:

- **# (numeral):** todo lo que sigue es un comentario. Python lo ignora al ejecutar. Sirve para que el programador explique el codigo.
- **import tkinter as tk:** carga la libreria de ventanas graficas. El as tk es un alias para no escribir tkinter completo cada vez.
- **from tkinter import messagebox:** importa un modulo especifico de tkinter que permite mostrar ventanas de alerta emergentes.

TIP: tkinter ya viene incluida con Python. No necesitas instalar nada adicional.

PASO 8 Fase 2 - Definir los usuarios del sistema

Necesitamos una lista de usuarios validos con sus contraseñas. Usaremos un diccionario de Python.

Agrega esto debajo de las lineas de import:

```
# Usuarios registrados en el sistema
# Formato: 'nombre_usuario': 'contrasena'
usuarios = {
    'admin': 'Admin2024!',
    'estudiante1': 'Seguridad1!',
    'estudiante2': 'Python2024!',
    'invitado': 'Invitado1!'
}

# Maximo de intentos permitidos antes de bloquear
MAX_INTENTOS = 3

# Contador de intentos (lista para poder modificarlo
# desde dentro de las funciones)
intentos = [0]
```

Conceptos clave:

- **usuarios = { }**: un diccionario que guarda pares clave:valor. La clave es el nombre de usuario y el valor es la contraseña.
- **MAX_INTENTOS = 3**: una constante. El nombre en mayusculas indica que no debe cambiar durante la ejecucion.
- **intentos = [0]**: una lista con un solo numero. Permite modificar el contador desde dentro de las funciones.

ATENCION: En un sistema real NUNCA se guardan contraseñas en texto plano. Se usan funciones de hash como bcrypt o SHA-256. En este proyecto lo hacemos simple para enfocarnos en la logica.

PASO 9 Fase 3 - Funcion de verificacion de login

Esta funcion recibe el usuario y la contrasena ingresados y verifica si son correctos.

Agrega esto debajo del bloque de usuarios:

```
# Funcion que verifica si las credenciales son correctas
def verificar_login():
    usuario = entrada_usuario.get()
    contrasena = entrada_contrasena.get()

    # Si ya se agotaron los intentos, no hacer nada
    if intentos[0] >= MAX_INTENTOS:
        return

    # Verificar si el usuario y la contrasena son correctos
    if usuario in usuarios and usuarios[usuario] == contrasena:
        ventana_login.withdraw() # Ocultar ventana de login
        abrir_sesion(usuario)     # Abrir ventana de sesion
    else:
        intentos[0] += 1
        restantes = MAX_INTENTOS - intentos[0]
        if restantes > 0:
            etiqueta_error.config(
                text=f'Credenciales incorrectas. Intentos restantes:
{restantes}'
            )
        else:
            etiqueta_error.config(
                text='ACCESO BLOQUEADO: demasiados intentos fallidos.'
            )
            boton_login.config(state='disabled')
            entrada_usuario.config(state='disabled')
            entrada_contrasena.config(state='disabled')
```

Conceptos nuevos:

- **def verificar_login():** define una funcion. Todo el codigo indentado dentro de ella pertenece a la funcion.
- **.get():** obtiene el texto que el usuario escribio en un campo de entrada.
- **ventana_login.withdraw():** oculta la ventana de login temporalmente.
- **.config(state='disabled'):** desactiva un elemento de la interfaz para que no se pueda usar.
- **f'texto {variable}':** un f-string. Permite insertar el valor de una variable dentro de un texto.

ATENCION: La indentacion (espacios al inicio de cada linea dentro de la funcion) es obligatoria en Python. Usa exactamente 4 espacios o la tecla Tab.

PASO 10 Fase 4 - Funcion de sesion activa

Cuando el login es exitoso, esta funcion abre una nueva ventana que muestra la informacion del usuario y el boton de cerrar sesion.

Agrega esto debajo de la funcion anterior:

```
# Funcion que abre la ventana de sesion activa
def abrir_sesion(nombre_usuario):
    ventana_sesion = tk.Toplevel()
    ventana_sesion.title('Sesion Activa')
    ventana_sesion.geometry('400x300')
    ventana_sesion.configure(bg='#1e1e2e')
    ventana_sesion.resizable(False, False)

    # Mensaje de bienvenida
    tk.Label(ventana_sesion,
             text=f'Bienvenido/a, {nombre_usuario.upper()}!',
             font=('Arial', 16, 'bold'),
             bg='#1e1e2e', fg='#a6e3a1').pack(pady=30)
    tk.Label(ventana_sesion, text='Sesion iniciada correctamente.',
             font=('Arial', 11), bg='#1e1e2e', fg='#cdd6f4').pack()

    # Cuadro con datos del usuario
    frame_info = tk.Frame(ventana_sesion, bg='#313244', padx=20, pady=15)
    frame_info.pack(pady=20, padx=30, fill='x')
    tk.Label(frame_info, text=f'Usuario: {nombre_usuario}',
             font=('Arial', 10), bg='#313244', fg='#cdd6f4',
             anchor='w').pack(fill='x')
    tk.Label(frame_info, text='Rol: Usuario estandar',
             font=('Arial', 10), bg='#313244', fg='#cdd6f4',
             anchor='w').pack(fill='x')
    tk.Label(frame_info, text='Estado: Activo',
             font=('Arial', 10), bg='#313244', fg='#a6e3a1',
             anchor='w').pack(fill='x')

    # Funcion interna para cerrar sesion
    def cerrar_sesion():
        ventana_sesion.destroy()
        ventana_login.deiconify()
        entrada_usuario.delete(0, tk.END)
        entrada_contrasena.delete(0, tk.END)
        etiqueta_error.config(text='')
        intentos[0] = 0
        boton_login.config(state='normal')
        entrada_usuario.config(state='normal')
        entrada_contrasena.config(state='normal')

    # Boton de cerrar sesion
    tk.Button(ventana_sesion, text='Cerrar Sesion',
             command=cerrar_sesion,
             bg='#f38ba8', fg='white',
             font=('Arial', 11, 'bold'))
```

```
padx=20, pady=8, bd=0,  
cursor='hand2') .pack(pady=10)
```

Conceptos nuevos:

- **tk.Toplevel():** crea una nueva ventana independiente.
- **.geometry('400x300'):** define el tamaño de la ventana en píxeles (ancho x alto).
- **.configure(bg='#1e1e2e'):** establece el color de fondo con un código hexadecimal.
- **tk.Label(...):** crea una etiqueta de texto en la ventana.
- **.pack(pady=30):** coloca el elemento en la ventana con un margen vertical de 30 píxeles.
- **tk.Frame(...):** crea un contenedor para agrupar otros elementos.
- **tk.Button(...):** crea un botón. El parámetro command indica que función ejecutar al hacer clic.
- **.destroy():** cierra y elimina una ventana.
- **.deiconify():** muestra una ventana que estaba oculta.
- **.delete(0, tk.END):** borra todo el texto de un campo de entrada.

PASO 11 Fase 5 - Construir la ventana principal

Esta es la parte que crea la ventana de login que el usuario vera al abrir el programa. Agrega todo esto al final del archivo.

Agrega esto al final del archivo login.py:

```
# =====
# CONSTRUCCION DE LA VENTANA PRINCIPAL
# =====
ventana_login = tk.Tk()
ventana_login.title('Sistema de Login - CIB-07')
ventana_login.geometry('420x480')
ventana_login.configure(bg='#1e1e2e')
ventana_login.resizable(False, False)

# Encabezado
frame_header = tk.Frame(ventana_login, bg='#313244', pady=20)
frame_header.pack(fill='x')
tk.Label(frame_header, text='SISTEMA DE LOGIN',
         font=('Arial', 18, 'bold'), bg='#313244', fg='#cba6f7').pack()
tk.Label(frame_header, text='CIB-07 Modulo 7 - Ciberseguridad',
         font=('Arial', 10), bg='#313244', fg='#6c7086').pack()

# Formulario
frame_form = tk.Frame(ventana_login, bg='#1e1e2e', padx=40, pady=30)
frame_form.pack(fill='both', expand=True)
tk.Label(frame_form, text='Nombre de usuario', font=('Arial', 11),
         bg='#1e1e2e', fg='#cdd6f4', anchor='w').pack(fill='x')
entrada_usuario = tk.Entry(frame_form, font=('Arial', 12), bg='#313244',
                           fg='#cdd6f4', insertbackground='white', bd=0,
                           highlightthickness=1, highlightcolor='#cba6f7',
                           highlightbackground='#45475a')
entrada_usuario.pack(fill='x', ipady=8, pady=(4, 16))
tk.Label(frame_form, text='Contraseña', font=('Arial', 11),
         bg='#1e1e2e', fg='#cdd6f4', anchor='w').pack(fill='x')
entrada_contrasena = tk.Entry(frame_form, font=('Arial', 12),
                              bg='#313244',
                              fg='#cdd6f4', insertbackground='white', bd=0, show='*',
                              highlightthickness=1, highlightcolor='#cba6f7',
                              highlightbackground='#45475a')
entrada_contrasena.pack(fill='x', ipady=8, pady=(4, 24))
boton_login = tk.Button(frame_form, text='Iniciar Sesión',
                        command=verificar_login, bg='#cba6f7', fg='#1e1e2e',
                        font=('Arial', 12, 'bold'), padx=20, pady=10, bd=0,
                        cursor='hand2')
boton_login.pack(fill='x')
etiqueta_error = tk.Label(frame_form, text='', font=('Arial', 10),
                          bg='#1e1e2e', fg='#f38ba8', wraplength=320)
etiqueta_error.pack(pady=(16, 0))

# Pie con usuarios de prueba
frame_pie = tk.Frame(ventana_login, bg='#181825', pady=12)
```



```
frame_pie.pack(fill='x', side='bottom')
tk.Label(frame_pie,
        text='Usuarios de prueba: admin / Admin2024! | invitado /
Invitado1!',
        font=('Arial', 8), bg='#181825', fg='#45475a').pack()

# Permitir iniciar sesion con la tecla Enter
ventana_login.bind('<Return>', lambda e: verificar_login())

# Iniciar el programa
ventana_login.mainloop()
```

Elementos clave de la interfaz:

- **tk.Tk():** crea la ventana principal del programa. Solo puede haber una por programa.
- **tk.Entry(show='*')**: el parametro show='*' hace que el campo muestre asteriscos en lugar del texto real.
- **tk.Label(fg='#f38ba8')**: el parametro fg define el color del texto. Este rojo se usa para los mensajes de error.
- **ventana_login.bind('<Return>', ...)**: vincula la tecla Enter al boton de login.
- **ventana_login.mainloop()**: inicia el ciclo principal que mantiene la ventana abierta. Siempre va al final.

PARTE 4 - Código completo

A continuación está el código completo. Usalo para verificar que tu archivo login.py tenga todo correcto. Respeta exactamente los espacios de indentación.

```
# =====
# SISTEMA DE LOGIN CON INTERFAZ GRAFICA
# CIB-07 Modulo 7 - Ciberseguridad
# =====
import tkinter as tk

usuarios = {
    'admin': 'Admin2024!',
    'estudiante1': 'Seguridad1!',
    'estudiante2': 'Python2024!',
    'invitado': 'Invitado1!'
}
MAX_INTENTOS = 3
intentos = [0]

def verificar_login():
    usuario = entrada_usuario.get()
    contrasena = entrada_contrasena.get()
    if intentos[0] >= MAX_INTENTOS:
        return
    if usuario in usuarios and usuarios[usuario] == contrasena:
        ventana_login.withdraw()
        abrir_sesion(usuario)
    else:
        intentos[0] += 1
        restantes = MAX_INTENTOS - intentos[0]
        if restantes > 0:
            etiqueta_error.config(
                text=f'Credenciales incorrectas. Intentos restantes:
{restantes}')
        else:
            etiqueta_error.config(
                text='ACCESO BLOQUEADO: demasiados intentos fallidos.')
            boton_login.config(state='disabled')
            entrada_usuario.config(state='disabled')
            entrada_contrasena.config(state='disabled')

def abrir_sesion(nombre_usuario):
    ventana_sesion = tk.Toplevel()
    ventana_sesion.title('Sesion Activa')
    ventana_sesion.geometry('400x300')
    ventana_sesion.configure(bg='#1e1e2e')
    ventana_sesion.resizable(False, False)
    tk.Label(ventana_sesion,
              text=f'Bienvenido/a, {nombre_usuario.upper()}!',
              font=('Arial', 16, 'bold'), bg='#1e1e2e',
              fg='#a6e3a1').pack(pady=30)
    tk.Label(ventana_sesion, text='Sesion iniciada correctamente.',
              font=('Arial', 11), bg='#1e1e2e', fg='#cdd6f4').pack()
```

```

        frame_info = tk.Frame(ventana_sesion, bg='#313244', padx=20, pady=15)
        frame_info.pack(pady=20, padx=30, fill='x')
        tk.Label(frame_info, text=f'Usuario: {nombre_usuario}',
                  font=('Arial', 10), bg='#313244', fg='#cdd6f4',
                  anchor='w').pack(fill='x')
        tk.Label(frame_info, text='Rol: Usuario estandar',
                  font=('Arial', 10), bg='#313244', fg='#cdd6f4',
                  anchor='w').pack(fill='x')
        tk.Label(frame_info, text='Estado: Activo',
                  font=('Arial', 10), bg='#313244', fg='#a6e3a1',
                  anchor='w').pack(fill='x')
        def cerrar_sesion():
            ventana_sesion.destroy()
            ventana_login.deiconify()
            entrada_usuario.delete(0, tk.END)
            entrada_contrasena.delete(0, tk.END)
            etiqueta_error.config(text='')
            intentos[0] = 0
            boton_login.config(state='normal')
            entrada_usuario.config(state='normal')
            entrada_contrasena.config(state='normal')
        tk.Button(ventana_sesion, text='Cerrar Sesion', command=cerrar_sesion,
                  bg='#f38ba8', fg='white', font=('Arial', 11, 'bold'),
                  padx=20, pady=8, bd=0, cursor='hand2').pack(pady=10)

    ventana_login = tk.Tk()
    ventana_login.title('Sistema de Login - CIB-07')
    ventana_login.geometry('420x480')
    ventana_login.configure(bg='#1e1e2e')
    ventana_login.resizable(False, False)
    frame_header = tk.Frame(ventana_login, bg='#313244', pady=20)
    frame_header.pack(fill='x')
    tk.Label(frame_header, text='SISTEMA DE LOGIN', font=('Arial', 18,
        'bold'),
            bg='#313244', fg='#cba6f7').pack()
    tk.Label(frame_header, text='CIB-07 Modulo 7 - Ciberseguridad',
            font=('Arial', 10), bg='#313244', fg='#6c7086').pack()
    frame_form = tk.Frame(ventana_login, bg='#1e1e2e', padx=40, pady=30)
    frame_form.pack(fill='both', expand=True)
    tk.Label(frame_form, text='Nombre de usuario', font=('Arial', 11),
            bg='#1e1e2e', fg='#cdd6f4', anchor='w').pack(fill='x')
    entrada_usuario = tk.Entry(frame_form, font=('Arial', 12), bg='#313244',
            fg='#cdd6f4', insertbackground='white', bd=0,
            highlightthickness=1, highlightcolor='#cba6f7',
            highlightbackground='#45475a')
    entrada_usuario.pack(fill='x', ipady=8, pady=(4, 16))
    tk.Label(frame_form, text='Contrasena', font=('Arial', 11),
            bg='#1e1e2e', fg='#cdd6f4', anchor='w').pack(fill='x')
    entrada_contrasena = tk.Entry(frame_form, font=('Arial', 12),
            bg='#313244',
            fg='#cdd6f4', insertbackground='white', bd=0, show='*',
            highlightthickness=1, highlightcolor='#cba6f7',
            highlightbackground='#45475a')
    entrada_contrasena.pack(fill='x', ipady=8, pady=(4, 24))
    boton_login = tk.Button(frame_form, text='Iniciar Sesion',
            command=verificar_login, bg='#cba6f7', fg='#1e1e2e',

```

```
        font=('Arial', 12, 'bold'), padx=20, pady=10, bd=0,
        cursor='hand2')
    boton_login.pack(fill='x')
    etiqueta_error = tk.Label(frame_form, text='', font=('Arial', 10),
        bg='#1e1e2e', fg='#f38ba8', wraplength=320)
    etiqueta_error.pack(pady=(16, 0))
    frame_pie = tk.Frame(ventana_login, bg='#181825', pady=12)
    frame_pie.pack(fill='x', side='bottom')
    tk.Label(frame_pie,
        text='Usuarios de prueba: admin / Admin2024! | invitado /
    Invitado1!',
        font=('Arial', 8), bg='#181825', fg='#45475a').pack()
    ventana_login.bind('<Return>', lambda e: verificar_login())
    ventana_login.mainloop()
```

PARTE 5 - Ejecutar y probar el programa

PASO 12 Ejecutar el programa

1. Asegurate de haber guardado el archivo login.py con Ctrl + S.
2. Haz clic en el boton de "play" arriba a la derecha en VS Code.
3. Aparecera la ventana de login con fondo oscuro y encabezado morado, exactamente como en las imagenes del inicio de esta guia.

ATENCION: Si aparece un error que dice 'No module named tkinter', escribe en la terminal de VS Code: `pip install tk --break-system-packages`

PASO 13 Pruebas obligatorias

Realiza los siguientes escenarios y toma una captura de pantalla de cada resultado:

#	Escenario	Datos a ingresar	Resultado esperado
1	Login exitoso	admin Admin2024!	Se oculta la ventana de login y aparece la ventana de bienvenida.
2	Contraseña incorrecta	admin 12345	Mensaje en rojo con intentos restantes.
3	Usuario inexistente	fantasma cualquiera	Mensaje en rojo con intentos restantes.
4	Bloqueo por 3 intentos	3 intentos incorrectos	Mensaje ACCESO BLOQUEADO, campos y boton desactivados.
5	Cerrar sesion	Login exitoso, luego clic en Cerrar Sesion	Vuelve la ventana de login limpia y activa.

TIP: Para tomar captura en Windows presiona Windows + Shift + S, arrastra para seleccionar el area y pega la imagen en un documento Word con Ctrl + V.

PARTE 6 - Que debes entregar

Sube los siguientes archivos a la plataforma del curso:

1. El archivo login.py con tu código completo y funcionando.
2. Un documento Word o PDF con las 5 capturas de pantalla de las pruebas, cada una etiquetada con su número.
3. En ese mismo documento responde las siguientes preguntas de reflexión (mínimo 3 oraciones por respuesta):

a)	¿Que es la autenticacion y como la implementa tu programa? <i>Respuesta:</i> Es el proceso de comprobar que una persona realmente es quien dice ser antes de darle acceso al sistema. En el programa se hace cuando el usuario ingresa su nombre de usuario y contraseña, y el sistema compara esos datos con los usuarios registrados. Si los datos son correctos, se abre la sesión; si son incorrectos, se cuenta el intento y después de 3 intentos fallidos el acceso queda bloqueado.
b)	¿Por que es un problema de seguridad guardar contraseñas en texto plano? ¿Como se resuelve en sistemas reales? <i>Respuesta:</i> Guardar las contraseñas en texto plano es un problema porque, si alguien logra acceder a la información del sistema, podría ver directamente las contraseñas de los usuarios. En un sistema real, las contraseñas se protegen usando funciones de hash, de manera que no se guarde directamente la contraseña original.
c)	¿Que mejora de seguridad le agregarías a este sistema si tuvieras mas tiempo? <i>Respuesta:</i> Agregaría un sistema para guardar las contraseñas de forma segura usando hash, en lugar de tenerlas directamente escritas en el código. También agregaría una opción para que el usuario pueda recuperar o cambiar su contraseña de forma segura.
d)	¿En que fase del SDL se deben tomar decisiones sobre el manejo de contraseñas y por que? <i>Respuesta:</i> Las decisiones sobre el manejo de las contraseñas se deben tomar desde la fase de diseño del SDL. Esto es importante porque desde el inicio se puede definir cómo se van a proteger las contraseñas y evitar problemas de seguridad después.

Criterios de evaluacion

Criterio	Peso	Calificacion
El archivo .py existe, corre sin errores y muestra la ventana de login.	30%	
Las 5 pruebas documentadas con capturas de pantalla correctamente etiquetadas.	25%	
El código incluye comentarios que explican cada sección.	15%	
Las 4 preguntas de reflexión respondidas con sus propias palabras.	30%	
TOTAL	100%	

Recuerda: El objetivo no es que el código sea perfecto, sino que lo entiendas. Si algo no funciona, documenta que intentaste hacer para resolverlo. Eso también se valora.